Bericht zur Programmieraufgabe (Taster-LED-Spiel)

Modul: Technische Informatik (Sommersemester 2026)

Aufgabe: Implementierung eines Simon-Says-Spiels auf Registerebene

Autor: Kai Kheiari

Matrikelnummer: 217324

- --

1. Systemübersicht

Dieser Bericht beschreibt die Implementierung eines Simon-Says-Spiels (Ähnlich wie Simon Says) auf dem STM32G431KB-Mikrocontroller. Das Spielprinzip basiert auf der Wiedergabe einer zufällig generierten Bitsequenz über eine LED, welche der Spieler anschließend fehlerfrei über zwei Taster replizieren muss.

Für den Hardwareaufbau wurden folgende Pinbelegungen auf dem Nucleo-Board gewählt:

- **PB8 (Ausgang):** Anschluss der grünen Benutzer-LED zur Anzeige der Blinksignale sowie zur Rückmeldung von Erfolg oder Misserfolg.
- **PA0 (Eingang):** Anschluss von Taster 1 zur Eingabe des Bitwerts `1` (langes LED-Signal). Der Pin ist intern mit einem Pull-Up-Widerstand beschaltet und schaltet aktiv gegen Masse (GND).
- **PA2 (Eingang):** Anschluss von Taster 2 zur Eingabe des Bitwerts `0` (kurzes LED-Signal). Auch dieser Pin nutzt den internen Pull-Up-Widerstand und schaltet gegen Masse.
- **PA3 (Analogeingang):** Ein unbeschalteter Pin (Floating Pin), der als Rauschquelle für den Analog-Digital-Umsetzer (ADC1) dient, um echte Zufallszahlen für die Sequenzgenerierung zu erzeugen.

- --

2. Register-Konfiguration

Die softwareseitige Ansteuerung erfolgt ohne HAL-Abstraktionsschichten direkt über die Hardwareregister des Mikrocontrollers. Die Initialisierung ist in folgende Funktionsbereiche unterteilt:

2.1 GPIO-Konfiguration

Über das Peripheral-Clock-Enable-Register des Systemtaktmoduls (`RCC->AHB2ENR`) wird das Taktsignal für die GPIO-Ports A und B aktiviert. Der Pin PB8 wird im Register `GPIOB->MODER` als digitaler Ausgang definiert. Die Pins PA0 und PA2 werden als digitale Eingänge konfiguriert, wobei im Register `GPIOA->PUPDR` die internen Pull-Up-Widerstände aktiviert werden.

2.2 Interrupt-Steuerung

Die Tastereingaben werden ereignisgesteuert über externe Interrupts verarbeitet. Hierzu wird das Taktsignal des Systemkonfigurations-Controllers (`SYSCFG`) aktiviert und der Multiplexer in `SYSCFG->EXTICR[0]` so eingestellt, dass die Leitungen EXTI0 (für PA0) und EXTI2 (für PA2) auf Port A verweisen. Die Register `EXTI->FTSR1` und `EXTI->RTSR1` werden so konfiguriert, dass Interrupts nur bei einer fallenden Signalflanke (Drücken des Tasters gegen GND) ausgelöst werden. Die Freigabe der Interrupts im NVIC erfolgt über `NVIC_EnableIRQ`.

2.3 Timer-Konfiguration

Zur Messung von Zeitintervallen (LED-Leuchtdauer, Pausenzeiten) wird der Basis-Timer TIM6 verwendet. Der Vorteiler (`TIM6->PSC`) ist auf 15999 gesetzt, sodass der Timer bei einem Systemtakt von 16 MHz mit einer Frequenz von 1 kHz (Intervall von 1 ms) zählt. Bei jedem Überlauf wird ein Interrupt ausgelöst, der im Handler `TIM6_DAC_IRQHandler` eine globale Variable `state_timer` dekrementiert.

2.4 Rausch-Generator (ADC)

Zur Erzeugung zufälliger Blinkmuster wird wie im vorlesungs beispiel, die Rauschspannung an Pin PA3 über den ADC1 digitalisiert. Die Initialisierung aktiviert den internen Spannungsregler des ADC und konfiguriert Kanal 4 für die Wandlung. Für eine 32-Bit-Zufallszahl werden 32 aufeinanderfolgende Wandlungen gestartet. Aus dem Datenregister `ADC1->DR` wird jeweils das niederwertigste Bit (LSB) extrahiert und zu einem 32-Bit-Wert zusammengesetzt.

- --

3. Zustandsdiagramm

Der Ablauf des Spiels ist als nicht-blockierende Zustandsmaschine in der Hauptschleife der Anwendung implementiert. Die Steuerung erfolgt über das globale Enum `GameState_t`.

```
stateDiagram-v2
    [*] --> STATE_INIT
    STATE_INIT --> STATE_START_LEVEL : HW-Initialisierung abgeschlossen

    STATE_START_LEVEL --> STATE_SHOW_SEQUENCE : 1s Vorlaufzeit abgelaufen / Sequenzlaenge = Level + 4 & Zufallsbits aus ADC1 generieren

    STATE_SHOW_SEQUENCE --> STATE_WAIT_INPUT : LED-Ausgabe beendet / Taster freischalten (EXTI aktiv)

    STATE_WAIT_INPUT --> STATE_PLAYING_INPUT : Taster gedrueckt / LED an (Feedback-Signal starten) & EXTI sperren

    STATE_PLAYING_INPUT --> STATE_DEBOUNCE_RELEASE : Feedback-LED erloschen / LED ausschalten
```

```
STATE_DEBOUNCE_RELEASE --> STATE_WAIT_INPUT : 50ms abgelaufen & Bit korrekt (Sequenz unvollstaendig) / EXTI freischalten
STATE_DEBOUNCE_RELEASE --> STATE_LEVEL_SUCCESS : 50ms abgelaufen & gesamte Sequenz korrekt
STATE_DEBOUNCE_RELEASE --> STATE_LEVEL_FAILURE : 50ms abgelaufen & falsches Bit eingegeben

STATE_LEVEL_SUCCESS --> STATE_START_LEVEL : Erfolgs-Blinken beendet (3s Pause / Level++)
STATE_LEVEL_FAILURE --> STATE_START_LEVEL : Fehler-Blinken beendet (3s Pause / Level = 1)
```

Beschreibung der Zustände:

(Zeitwerte Können sich im verlauf von debugging/Entwicklilunh leicht geändert haben,ich hafte nicht für leichte Abweischungen)

- **`STATE_INIT`:** Initialisierung der Hardwareregister, Clocks und Timer-Interrupts. Anschließend automatischer Wechsel in das erste Level.
- **`STATE_START_LEVEL`:** Bestimmung der aktuellen Bitlänge (Level + 4) und Speichern der gewürfelten Bitsequenz. Vor dem Abspielen wird eine Verzögerung von 1s eingehalten.
- **`STATE_SHOW_SEQUENCE`:** Ausgabe der Sequenz über PB8. Ein `0`-Bit schaltet die LED für 0,35s ein, ein `1`-Bit für 1,0s. Die Pause zwischen den Signalen beträgt 0,2s. Die Taster-Interrupts sind während dieser Phase deaktiviert. Nach der Ausgabe folgt eine Pause von 1,5s, bevor die Taster freigeschaltet werden.
- **`STATE_WAIT_INPUT`:** Das Hauptprogramm versetzt den Core mittels des Befehls `__WFI()` in den Schlafmodus, um Energie zu sparen. Das System wacht erst bei einem Tastendruck über den EXTI-Handler auf.
- **`STATE_PLAYING_INPUT`:** Visuelle Rückmeldung der Eingabe für den Spieler. Drücken von PA0 (Eingabe `1`) schaltet die LED für 1,0s ein, Drücken von PA2 (Eingabe `0`) für 0,35s. Zur Vermeidung von Mehrfacheingaben werden die EXTI-Leitungen gesperrt.
- **`STATE_DEBOUNCE_RELEASE`:** Nach dem Erlöschen des Feedback-Signals wartet das System 50 ms, um das Prellen der Tasterkontakte beim Loslassen softwareseitig auszufiltern. Anschließend wird geprüft, ob das eingegebene Bit korrekt war.
- **`STATE_LEVEL_SUCCESS`:** Die LED blinkt zweimal lang (je 1,0s an, 0,2s aus) zur Signalisierung des Levelerfolgs. Das Level wird inkrementiert (maximal Level 5) und nach 3s startet das nächste Level.
- **`STATE_LEVEL_FAILURE`:** Die LED blinkt dreimal schnell (je 0,15s an, 0,15s aus) zur Fehlersignalisierung. Das Spiel setzt das Level auf 1 zurück und startet nach 3s neu.
- --

4. Kompilierung

Das Projekt verfügt über ein integriertes Makefile, welches sowohl den Übersetzungsvorgang für die Zielhardware (ARM Cortex-M4) als auch für den Desktop-Simulator (Linux/x86) steuert (mehr zu dem Simulator weiter Unten)

4.1 Target-Build

Zur Übersetzung des Quellcodes für die ARM-Zielplattform wird das GNU-Toolchain-Präfix `arm-none-eabi-` verwendet.

- **Kompilieren:**

```
make
```

Der Befehl generiert die Binärdateien im Unterordner `build/`.

- **Flashen:**

```
make flash
```

Dieser Befehl übersetzt das Projekt und lädt es anschließend mittels OpenOCD auf das angeschlossene Nucleo-Board hoch.

4.2 Desktop-Build

Für Testzwecke ohne Hardware-Board kann eine emulierte Version des Spiels direkt im Linux-Terminal ausgeführt werden (Für andere Systeme kann ich nicht unbedingt garantieren aber theoretisch sollte auch andere terminals und betriebsysteme möglich sein)

- **Kompilieren:**

```
make desktop
```

Dies kompiliert die Firmware-Logik zusammen mit dem Desktop-Treiber zu einem ausführbaren x86-Programm namens `test_game`.

- **Ausführen:**

```
./test_game
```

Das Terminal wird gelöscht und zeigt eine interaktive Oberfläche zur Eingabe über die Tasten `s` (short / kurz) und `l` (long / lang). Ein Debug-Menü kann über `d` + Enter aufgerufen werden.

- **Aufräumen:**

```
make clean
```

Entfernt alle Build-Artefakte des Mikrocontroller-Projekts sowie die ausführbare Datei des Simulators.